Phishing Message Detection Tool

Mini Project Report

PCET & NMVPM's Nutan Maharashtra Institute of Engineering and Technology, Talegaon Dabhade, Pune

An Autonomous Institute affiliated to Savitribai Phule Pune University (SPPU), Accredited by NAAC & NBA

Program: Computer Engineering | **Semester:** III
Course Code & Name: CE25VSEC-209A, Scientific Computing with Python
Batch: S11 | **Division:** D

Submitted by:

Sr. No.	Roll No.	Student Name	PRN
1	SECD139	Kalyani Jitendra Patil	U251A160
2	SECD136	Vedashree Tukaram Parab	U251A150

Faculty Coordinator: Prof. Kavyashree H.N

Academic Year: 2026–27

Acknowledgement

We would like to express our sincere gratitude to our Faculty Coordinator, Prof. Kavyashree H.N, for her guidance, encouragement and valuable suggestions throughout this mini project. Her feedback helped us to shape the idea of the Phishing Message Detection Tool into a well-structured, object-oriented Python application.

We are also thankful to the Head of the Computer Engineering Department and to the management and staff of Nutan Maharashtra Institute of Engineering and Technology for providing the facilities and a supportive learning environment. Finally, we thank our friends and families for their constant motivation.

Kalyani Jitendra Patil
Vedashree Tukaram Parab

Abstract

Phishing is one of the most widespread cyber-attacks today. Fraudsters send fake SMS and e-mail messages that appear to come from banks, courier companies, government offices or well-known brands. These messages create fear or greed and push the receiver to click a malicious link or share confidential data such as OTPs, passwords and card details. Most ordinary users do not have the technical knowledge to separate a genuine message from a phishing message.

This mini project presents the **Phishing Message Detection Tool**, a menu-driven, offline, console-based application written in Python 3. The tool accepts an SMS or e-mail text, validates the input, pre-processes the text, extracts every URL using regular expressions and analyses both the text and the links for known phishing indicators. Indicators such as urgent or threatening words, requests for OTP or KYC, IP-address links, shortened links, the '@' symbol in a URL, missing HTTPS, suspicious domain extensions and fake brand names each add weighted points to a risk score. Depending on the final score, the message is classified as **SAFE**, **SUSPICIOUS** or **PHISHING**, and the reasons are displayed along with safety advice.

The project is designed around core object-oriented concepts: classes, inheritance (SMSMessage and EmailMessage derived from Message), polymorphism, encapsulation, custom exceptions and file handling. Every scan is saved in a log file (scan_history.txt) so that the user can view the history later. The result is a simple, extendable and educational tool that gives an instant warning before a user clicks a link or shares sensitive details.

Keywords: Phishing, SMS, E-mail, URL analysis, regular expressions, risk score, Python, OOP, exception handling, file handling.

Table of Contents

1. Project Details
2. Student Details
3. Introduction
4. Problem Statement
5. Need of the Project
6. Objectives
7. Scope and Limitations of the Project
8. Literature Survey and Background
9. Proposed System
10. Major Modules / Features
11. Requirement Analysis
12. Concepts to be Used
13. Proposed Technology
14. System Design
15. Proposed System Workflow
16. Detailed Algorithm
17. Implementation (Python Code)
18. Testing and Test Cases
19. Sample Output
20. Results and Discussion
21. Advantages, Limitations and Future Scope
22. Expected Outcome
23. Tentative Project Plan
24. Conclusion
25. References
26. Declaration

1. Project Details

Project Title: Phishing Message Detection Tool

Project Domain: Cyber Security / Text Analysis (Python-based console application)

Program: Computer Engineering

Semester: III

Course Code & Name: CE25VSEC-209A, Scientific Computing with Python

Batch: S11

Division: D

Type of Project: Mini Project (Object-Oriented, Console Application)

Institute: PCET & NMVPM's Nutan Maharashtra Institute of Engineering and Technology (NMIET), Talegaon Dabhade, Pune

2. Student Details

Sr. No.	Roll No.	Student Name	PRN
1	SECD139	Kalyani Jitendra Patil	U251A160
2	SECD136	Vedashree Tukaram Parab	U251A150

3. Introduction

3.1 Background

Mobile phones and e-mail have become the main channels for banking alerts, delivery updates, government notices and personal communication. Attackers take advantage of this trust. In a phishing attack, the attacker pretends to be a trusted entity and sends a message that contains a warning, an offer or a request. The message normally carries a link to a fake website that looks like the real one, or asks the receiver to reply with confidential information.

Phishing through SMS is often called *smishing*, and phishing through e-mail is simply called *e-mail phishing*. Both rely on social engineering: instead of breaking into a computer system, the attacker tricks the human user. Because of this, technical security measures alone cannot fully stop the attack. The user must be able to recognise a suspicious message, and a tool that helps the user take that decision is very useful.

3.2 Common Types of Phishing Messages

The following themes are seen again and again in phishing messages:

- **Bank and KYC messages:** "Your account will be blocked today. Update your KYC immediately."
- **Courier and delivery messages:** "Your parcel is on hold. Pay a small fee to release it."
- **Prize and lottery messages:** "Congratulations! You have won a prize. Click here to claim."
- **Government and tax messages:** "Your refund is ready. Verify your details to receive it."
- **Account security messages:** "Unusual login detected. Verify your password now."
- **OTP and card messages:** "Share the OTP you received to complete the cancellation."

3.3 Why a Detection Tool Is Needed

Phishing messages are written to create panic or excitement so that the user acts without thinking. A tool that reads the message calmly, counts the warning signs and explains them in simple language can give the user the moment of thought that the attacker wants to avoid. The tool in this project does exactly this, and it does so completely offline, so the text of a private

message never leaves the user's computer.

3.4 Overview of the Project

The Phishing Message Detection Tool is a menu-driven Python program. The user chooses the option *Check Message*, selects the message type (SMS or e-mail), optionally enters the sender, and pastes the message text. The program validates the input, cleans the text, extracts links, runs the detection rules and prints a verdict with reasons. All results are appended to a log file and can be viewed from the *View History* option.

4. Problem Statement

Phishing is a cyber-attack in which fraudsters send fake SMS and e-mail messages that look like they come from banks, courier companies, government offices or well-known brands. These messages create fear or greed and trick users into clicking malicious links or sharing OTPs, passwords and card details. Most ordinary users cannot easily tell a genuine message from a phishing message.

The problem is to design and develop an object-oriented Python tool that accepts a message, analyses its text and links, detects phishing indicators, and tells the user whether the message is Safe, Suspicious or Phishing, along with the reasons.

4.1 Detailed Problem Analysis

The problem can be divided into the following smaller problems:

1. How can a message and its type be accepted safely from the user, and how can wrong input be rejected with a clear error?
2. How can the free-form text be converted into a form that is easy to analyse?
3. How can all links be found in the text, even when they are surrounded by punctuation?
4. Which words, phrases and link features are good indicators of phishing?
5. How can the indicators be combined into one number that can be compared with limits?
6. How can the decision be explained to the user so that the user also learns from it?
7. How can the results be stored and read back without losing data or crashing when the file is missing?

Each of these questions is solved by one or more classes and functions in the proposed system.

5. Need of the Project

- Phishing and fake-link messages are increasing every day and cause financial loss and loss of personal data.
- Common users have little technical knowledge to recognise suspicious messages and links.
- A simple, offline tool can give an instant warning before the user clicks a link or shares sensitive details.
- The project applies core OOP concepts – classes, inheritance, polymorphism, exception handling and file handling – to a real-life security problem.

5.1 Additional Points Supporting the Need

- **Speed of the attack:** Phishing messages ask the user to act within minutes or hours. A tool that answers in a second fits this time window.
- **Awareness through explanation:** Most tools only show a label such as "spam". This tool lists the exact reasons, so the user learns what a phishing message looks like and becomes more careful in the future.
- **Privacy:** Online scanners need the user to upload the message or link to a server. An offline tool keeps the data on the local machine.

- **No special resources:** The tool needs only a Python interpreter. It does not need internet, a database or a paid service.
- **Educational value:** For students, the project is a practical example of how strings, regular expressions, dictionaries, classes and files work together in a real application.

6. Objectives

1. To study the common patterns and characteristics of phishing SMS and e-mail messages.
2. To design an object-oriented system in Python using classes, inheritance and polymorphism.
3. To detect suspicious keywords and unsafe URLs in a given message using pattern matching.
4. To calculate a risk score and classify the message as Safe, Suspicious or Phishing with clear reasons.
5. To store scan results in a file and handle invalid input and file errors using exception handling.

6.1 Secondary Objectives

- To keep the detection rules in dictionaries and lists so that new keywords, brands and shortener domains can be added without changing the program logic.
- To keep the design modular so that each class has one clear responsibility.
- To provide safety advice after every scan.
- To keep the user interface simple enough for a non-technical person.

7. Scope and Limitations of the Project

7.1 Scope

- The tool analyses the text of an SMS or e-mail typed or pasted by the user.
- It analyses the structure of each link found in the message (domain, scheme, extension) without opening it.
- It classifies the message into three levels and explains the reasons.
- It stores the history in a plain text file.
- It runs in the console on Windows, Linux and macOS.

7.2 Limitations

- The tool is rule-based. It does not learn from data and does not use machine learning.
- It does not connect to the internet, so it cannot check whether a link is listed in a live blacklist or how old a domain is.
- It does not read attachments, images or QR codes.
- A genuine message that contains words such as OTP or account (for example a real bank OTP alert) may receive some risk points. The thresholds are chosen so that one or two such words give SAFE.
- Keywords are in English. Messages in other languages or in Hinglish are only partly covered unless the keyword list is extended.
- The result is advice, not a guarantee. A new attack that uses none of the known indicators may be shown as SAFE.

8. Literature Survey and Background

8.1 Approaches to Phishing Detection

Phishing detection techniques are usually grouped into three families.

1. List-based (blacklist / whitelist) detection. The link is compared with a list of known malicious or known safe sites. This is fast and accurate for known sites, but it fails for new sites because a phishing site often lives for only a few hours and may not yet be on any list.

2. Heuristic / rule-based detection. The message and link are checked against a set of rules that describe how phishing messages usually look: urgent language, requests for confidential data, links with IP addresses, very long or shortened URLs, a brand name placed inside an unrelated domain, and so on. These rules need no training data and can explain their decision. Their weakness is that attackers can change their style and the rules must be updated.

3. Machine-learning-based detection. A model is trained on thousands of labelled messages or URLs and learns to separate the classes. This can find patterns that humans miss, but it needs a large and up-to-date dataset, more computing power and is harder to explain.

This project follows the **heuristic, rule-based approach** because it is transparent, works offline, needs no dataset and is the best fit for demonstrating object-oriented programming and pattern matching in a Scientific Computing with Python course.

8.2 Known Phishing Indicators

Public awareness material and the reports published by the Anti-Phishing Working Group (APWG) describe several recurring features of phishing messages. The ones used in this project are summarised below.

Category	Indicator	Why it is suspicious
Language	Urgent or threatening words (urgent, immediately, blocked, suspended)	Creates fear so that the user acts without thinking
Language	Greed words (prize, lottery, winner, free gift)	Creates excitement and lowers caution
Request	OTP, PIN, CVV, password, KYC	Genuine organisations do not ask for these through a link or reply
URL	IP address instead of a domain name	Real brands use names, not numbers
URL	Shortened link (bit.ly, tinyurl)	Hides the real destination
URL	'@' symbol inside the link	Text before '@' is ignored by browsers and can hide the real host
URL	No HTTPS	Genuine login and payment pages use encrypted connections
URL	Brand name inside a different domain	Imitates a trusted brand (for example sbi-kyc-update.xyz)
URL	Unusual domain extension (.xyz, .top, .tk)	Often cheap or free and widely abused
Sender	Free e-mail address or ordinary mobile number claiming to be a bank	Banks use official domains or registered sender IDs

8.3 Role of Python

Python provides ready-made modules that suit this problem. The **re** module handles pattern matching, **urllib.parse** splits a URL into scheme, host and path, **datetime** gives the time for the log, and the built-in file functions handle the history. Dictionaries and lists hold the rules. Because of this, the whole tool can be built using only the standard library.

9. Proposed System

The proposed system is a menu-driven Python application. The user enters an SMS or e-mail text. The system first validates the input, then pre-processes the text and extracts all URLs. A message object (SMSMessage or EmailMessage, both derived from the base class Message) is created and passed to the PhishingDetector class.

The detector checks the text for suspicious keywords and phrases and asks the URLAnalyzer class to check each link for warning signs such as an IP address in the URL, shortened links, the '@' symbol, missing HTTPS and fake brand names. Every indicator adds weighted points to a risk score. Based on the total score, the message is classified as SAFE, SUSPICIOUS or PHISHING. The result with reasons is displayed to the user and saved in a log file, which can be viewed later from the history option.

9.1 Working in Steps

1. The main menu is displayed with three options: Check Message, View History and Exit.
2. For *Check Message*, the user selects the type (SMS or E-mail), enters an optional sender and types the message.
3. The validator checks the type and the message length. If the input is wrong, a custom exception is raised and an error message is shown. The program returns to the menu.
4. A Message object of the proper subclass is created. Its constructor pre-processes the text and extracts the links.
5. The PhishingDetector scans the cleaned text against the keyword dictionary and the OTP/card request pattern.
6. For each extracted link, the URLAnalyzer returns points and reasons.
7. The Message subclass adds its own sender-based points (polymorphism).
8. The total score is compared with the thresholds and the label is selected.
9. A ScanResult object is displayed with reasons and safety advice, and the LogManager appends it to scan_history.txt.
10. The menu is shown again until the user chooses Exit.

9.2 Design Principles Followed

- **Single responsibility:** each class does one job (represent a message, analyse a link, score a message, store results).
- **Data-driven rules:** keywords, weights, brands and shorteners are stored in dictionaries and lists at the top of the classes.
- **Fail-safe input handling:** all user input passes through the validator, and file operations are wrapped in try / except blocks.
- **Explainability:** every point added to the score is recorded as a reason string.

10. Major Modules / Features

Sr. No.	Module / Feature	Description
1	Message Input Module	Accepts an SMS or e-mail text typed by the user through a menu-driven console interface, along with the message type (SMS / E-mail) and optional sender details.
2	Input Validation Module	Checks that the message is not empty or too short and that the type is valid; raises custom exceptions (EmptyMessageError, InvalidMessageError) for wrong input.
3	Text Pre-processing & URL Extraction	Converts text to lowercase, removes extra spaces and symbols, and extracts all links from the message using regular expressions.
4	Phishing Detection Engine	Scans for suspicious keywords (urgent, verify, OTP, KYC, blocked, prize, lottery) and checks URLs for IP-address links, shortened links, '@' symbol, missing HTTPS and fake brand names.
5	Risk Scoring & Classification	Adds weighted points for every suspicious indicator and classifies the message as SAFE, SUSPICIOUS or PHISHING based on the final risk score.

Sr. No.	Module / Feature	Description
6	Report & Log Management	Displays the result with reasons and safety advice, saves every scan to a text file and lets the user view the previous scan history.

10.1 Module-wise Explanation

Module 1 – Message Input. This module shows the menu, reads the choice and collects the message type, sender and text. A multi-line message can be pasted as a single line or typed with a blank line to finish. The module only collects data; it does not judge it.

Module 2 – Input Validation. The module checks three conditions: the type must be SMS or EMAIL, the text must not be empty after removing spaces, and the text must have at least 10 characters. If the text is empty, EmptyMessageError is raised. If the type is wrong or the text is too short, InvalidMessageError is raised. The menu loop catches both and prints a friendly message.

Module 3 – Pre-processing and URL Extraction. The original text is first searched for links using a regular expression, because the cleaning step would damage them. The text is then converted to lowercase, symbols that are not useful are replaced by spaces and extra spaces are removed. The cleaned text is used for keyword matching.

Module 4 – Phishing Detection Engine. The engine has two parts. The *text part* looks for suspicious words and for sentences that ask the user to share an OTP, PIN, CVV or password. The *link part* is handled by the URLAnalyzer class and examines the domain, scheme and extension of each link.

Module 5 – Risk Scoring and Classification. Every matched rule has a weight. The weights are added to get the total risk score. If-elif conditions compare the score with two thresholds to give SAFE, SUSPICIOUS or PHISHING.

Module 6 – Report and Log Management. The report shows the type, sender, links found, score, label, list of reasons and a short safety advice. A one-line summary with date and time is appended to scan_history.txt. The history option reads and prints the file; if the file does not exist yet, a message is shown instead of an error.

11. Requirement Analysis

11.1 Functional Requirements

ID	Requirement
FR1	The system shall display a menu with the options Check Message, View History and Exit.
FR2	The system shall accept the message type (SMS or E-mail), optional sender and message text.
FR3	The system shall reject empty or too-short messages and invalid types with custom exceptions.
FR4	The system shall extract every URL from the message.
FR5	The system shall detect suspicious keywords and OTP/card-detail requests.
FR6	The system shall analyse every URL for IP address, shortener, '@' symbol, missing HTTPS, unusual extension and fake brand.
FR7	The system shall calculate a weighted risk score and classify the message as SAFE, SUSPICIOUS or PHISHING.
FR8	The system shall display the reasons for the decision and safety advice.
FR9	The system shall save every scan to scan_history.txt and display the history on request.

11.2 Non-Functional Requirements

Requirement	Description
Performance	A scan should complete in well under one second for a normal message.
Usability	Menu text and messages must be simple and clear for non-technical users.
Reliability	Wrong input or a missing log file must not crash the program.
Portability	The program must run on Windows, Linux and macOS with Python 3.x.
Maintainability	Rules are stored in dictionaries and lists; classes are small and documented.
Privacy	No message text is sent over the network.
Extensibility	New message types or rules can be added by creating a subclass or editing a dictionary.

11.3 Hardware Requirements

Component	Minimum requirement
Processor	Any dual-core processor
RAM	2 GB
Storage	50 MB free space
Display / Input	Standard monitor and keyboard

11.4 Software Requirements

Component	Requirement
Operating System	Windows 10 / 11 (also runs on Linux and macOS)
Language	Python 3.x
Editor / IDE	VS Code / PyCharm / IDLE
Libraries	re, urllib.parse, datetime (all in the Python standard library)

11.5 Assumptions and Constraints

- The user will paste the complete message including the link.
- The tool treats a message as English text.
- The log file is stored in the same folder as the program.

12. Concepts to be Used

Sr. No.	Concept	Proposed Application
1	Regular Expressions (Pattern Matching)	The re module is used to extract all links from the message and to detect patterns such as IP-address URLs, the '@' symbol and OTP / card-number requests.
2	String Processing	Message text is converted to lowercase, extra spaces and symbols are removed, and the text is split into words using string methods such as lower(), strip(), split() and replace().
3	URL Parsing & Analysis	The urllib.parse module splits each link into scheme, domain and path to check for missing HTTPS, shortened links (bit.ly, tinyurl), fake brand names and suspicious domain extensions.
4	Dictionaries & Lists (Data Structures)	A dictionary stores suspicious keywords with their risk weights, lists store shortened-URL domains and brand names, and a list holds the extracted links and the reasons found.

Sr. No.	Concept	Proposed Application
5	Keyword-based Rule Detection	The message is scanned against rules for suspicious words and phrases (urgent, verify, OTP, KYC, blocked, prize, lottery) and every matched rule is recorded as a reason.
6	Risk Scoring & Classification Logic	Weighted points are added for each indicator, and if-elif conditions compare the total score with threshold values to classify the message as SAFE, SUSPICIOUS or PHISHING.
7	Exception Handling & Input Validation	try / except / finally blocks with custom exceptions (EmptyMessageError, InvalidMessageError) validate the input and handle file errors such as FileNotFoundError.
8	File Handling	Scan results are written to and read from a log file (scan_history.txt) using open(), write(), read() and the with statement.

12.1 Object-Oriented Concepts in the Project

OOP Concept	Where it is used
Class and Object	Message, URLAnalyzer, PhishingDetector, ScanResult and LogManager are classes; each scan creates objects of them.
Inheritance	SMSMessage and EmailMessage inherit from the base class Message and the custom exceptions inherit from Exception.
Polymorphism	Both subclasses override get_type() and get_extra_score(); the detector calls the same method on any message and gets behaviour suited to that type.
Encapsulation	Rules, weights and thresholds are kept inside the classes that use them; other parts of the program use only the public methods.
Abstraction	The main program only calls scan(); the details of keyword matching and URL analysis are hidden inside the detector.
Exception handling	Custom exceptions signal bad input; try / except / finally protects menu input and file access.

13. Proposed Technology

Programming Language: Python 3.x

Compiler / IDE: Python (CPython) Interpreter; VS Code / PyCharm / IDLE

Operating System: Windows 10 / 11 (also runs on Linux and macOS)

13.1 Why Python?

- Simple, readable syntax that suits students and quick prototyping.
- Powerful built-in string, list and dictionary support.
- Standard library modules for regular expressions (re) and URL parsing (urllib.parse).
- Easy file handling with the with statement.
- Full support for object-oriented programming.
- Runs on all major operating systems without changes.

13.2 Python Modules Used

Module	Purpose in this project
re	Extracting URLs, detecting IP-address hosts and OTP/card requests, cleaning text
urllib.parse	Splitting each URL into scheme, host name and path
datetime	Adding date and time to every log entry
os (optional)	Checking whether the log file exists

14. System Design

14.1 System Architecture

The system follows a layered design. Each layer talks only to the layer next to it, which keeps the code easy to test and change.

Layer	Components	Responsibility
Presentation layer	Main menu, input prompts, result display	Talks to the user through the console
Validation layer	validate_input(), EmptyMessageError, InvalidMessageError	Rejects wrong input before any analysis
Data layer	Message, SMSMessage, EmailMessage	Holds the raw text, cleaned text, sender and extracted links
Analysis layer	PhishingDetector, URLAnalyzer	Applies keyword rules and URL rules and calculates the score
Result and storage layer	ScanResult, LogManager	Formats the report and reads/writes scan_history.txt

14.2 Class Design

Class	Type	Main attributes	Main methods
EmptyMessageError	Custom exception (inherits Exception)	message text	–
InvalidMessageError	Custom exception (inherits Exception)	message text	–
Message	Base class	raw_text, sender, clean_text, urls	preprocess(), extract_urls(), get_type(), get_extra_score()
SMSMessage	Derived from Message	inherited	get_type() and get_extra_score() overridden
EmailMessage	Derived from Message	inherited	get_type() and get_extra_score() overridden
URLAnalyzer	Utility class	SHORTENERS, BRANDS, BAD_TLDS, IP_PATTERN	analyze(url)
PhishingDetector	Engine class	KEYWORDS, thresholds, url_analyzer	scan(message), classify(score)
ScanResult	Result class	message type, sender, urls, score, label, reasons, time	display(), to_log_line()
LogManager	File class	file name	save(result), read_history()

Relationships:

- SMSMessage *is a* Message and EmailMessage *is a* Message (inheritance).
- PhishingDetector *has a* URLAnalyzer (composition).
- PhishingDetector *uses* Message and *produces* ScanResult (association).
- LogManager *stores* ScanResult objects as text lines.

14.3 Data Flow Overview

1. **User → Input module:** type, sender, raw text.
2. **Input module → Validator:** the same values are checked.
3. **Validator → Message object:** valid values create SMSMessage or EmailMessage, which stores the cleaned text and the list of URLs.
4. **Message object → PhishingDetector:** the detector reads the cleaned text and the URLs.
5. **PhishingDetector → URLAnalyzer:** each URL is sent and (points, reasons) are returned.
6. **PhishingDetector → ScanResult:** total score, label and reasons are packed.

7. **ScanResult → User and LogManager:** displayed on the screen and appended to scan_history.txt.

14.4 Data Structures Used

Structure	Example	Purpose
Dictionary	{"otp": 3, "urgent": 2}	Keyword and its risk weight
Dictionary	{"sbi": "sbi.co.in"}	Brand name and its genuine domain
List	["bit.ly", "tinyurl.com"]	Known URL shortener domains
Tuple	(".xyz", ".top", ".tk")	Suspicious domain extensions
List	urls, reasons	Extracted links and explanations
Compiled regex	URL_PATTERN, IP_PATTERN	Fast repeated pattern matching

14.5 Risk Score Table

Every indicator carries a weight. The weights were chosen so that strong signs (fake brand, IP address, OTP/KYC requests) count more than weak signs (the word "account").

Text indicators

Indicator	Points
urgent, immediately, verify, refund, click here, limited time	2 each
otp, kyc, blocked, suspended, prize, lottery, winner, password	3 each
account	1
Request to share/send OTP, PIN, CVV or password (without "do not share" or "never share")	3

URL indicators (per link)

Indicator	Points
Fake brand name in a domain that is not the official domain	4
IP address used as host	4
Shortened link	3
'@' symbol in the link	3
No HTTPS (http or www only)	2
Suspicious domain extension	2
Too many hyphens in the domain (3 or more)	1

Sender indicators (polymorphic)

Message type	Indicator	Points
SMS	Sender is an ordinary 10-digit mobile number and the text mentions a bank or account	2
E-mail	Sender uses a free e-mail domain (gmail.com, yahoo.com, outlook.com) and the text mentions a bank or account	2

Classification thresholds

Total score	Label
0 to 3	SAFE
4 to 9	SUSPICIOUS
10 and above	PHISHING

The thresholds are kept as class constants, so they can be tuned after testing.

15. Proposed System Workflow

The flowchart below is described step by step. It follows the diagram prepared in the project proposal.

1. **Start.**
2. **Display main menu:** 1. Check Message, 2. View History, 3. Exit.
3. **User choice?** History: read and display the log history, then return to the main menu. Exit: End. Check: continue to step 4.
4. **Enter SMS / E-mail message text** (input).
5. **Is input valid?** No: raise custom exception and show error message, then return to the main menu. Yes: continue.
6. **Pre-process text** (lowercase, clean, extract URLs using regex).
7. **Create message object** (SMSMessage / EmailMessage).
8. **Keyword and pattern analysis** (urgent words, OTP, KYC, prize, bank).
9. **URL analysis** (IP address, short link, '@' symbol, no HTTPS).
10. **Calculate risk score.**
11. **Risk score?** High: PHISHING. Medium: SUSPICIOUS. Low: SAFE.
12. **Display result with reasons.**
13. **Save result to log file**, then return to the main menu.

16. Detailed Algorithm

16.1 Main Program

1. Start.
2. Create the PhishingDetector and LogManager objects.
3. Repeat steps 4 to 12 until the user chooses Exit.
4. Show the menu and read the choice.
5. If the choice is 2, call read_history() and print it; go to step 3.
6. If the choice is 3, print a goodbye message and stop.
7. If the choice is 1, read the type, sender and message text.
8. Call validate_input(). If an exception is raised, print the error and go to step 3.
9. Create SMSMessage or EmailMessage depending on the type.
10. Call detector.scan(message) to get a ScanResult.
11. Call result.display().
12. Call log.save(result) and go to step 3.
13. Stop.

16.2 Scan Algorithm (PhishingDetector.scan)

1. Set score = 0 and reasons = [].
2. For each keyword and weight in the dictionary: if the keyword is found in the cleaned text, add the weight to score and add a reason.
3. If the cleaned text asks the user to share an OTP/PIN/CVV/password and does not contain "do not share" or "never share", add 3 points and a reason.
4. For each URL in the message: call URLAnalyzer.analyze(url); add the returned points and reasons.
5. Call message.get_extra_score() and add the returned points and reasons.
6. Call classify(score) to get the label.
7. If no reason was found, add "No phishing indicators found".

8. Return a ScanResult.

16.3 URL Analysis Algorithm (URLAnalyzer.analyze)

1. If the URL has no scheme, add "http://" so that urlparse can read it.
2. Parse the URL and get the host name (lowercase).
3. If the host matches the IP pattern, add 4 points.
4. If the host is in the shortener list, add 3 points.
5. If '@' is present in the URL, add 3 points.
6. If the original URL does not start with "https://", add 2 points.
7. If the host ends with a suspicious extension, add 2 points.
8. For each brand: if the brand name is in the host and the host is not the official domain or its sub-domain, add 4 points.
9. If the host has 3 or more hyphens, add 1 point.
10. Return the points and the list of reasons.

16.4 Time and Space Complexity

Let n be the number of characters in the message, k the number of keywords and u the number of links. Keyword matching takes O(n × k), and URL analysis takes O(u) because every link is short and the lists are small. For a normal SMS or e-mail these numbers are tiny, so the tool answers instantly. Memory use is O(n) for the text and the reasons.

17. Implementation (Python Code)

The complete program is given below. It uses only the Python standard library. Save it as `phishing_detector.py` and run it with `python phishing_detector.py`.

17.1 Custom Exceptions and Message Classes

```
import re
from datetime import datetime
from urllib.parse import urlparse


class EmptyMessageError(Exception):
    """Raised when the message is empty."""


class InvalidMessageError(Exception):
    """Raised when the type is wrong or the message is too short."""


class Message:
    """Base class for every message."""
    URL_PATTERN = re.compile(r'(?:https?://|www\.)[^\s<>"]+', re.I)
    FREE_DOMAINS = ("gmail.com", "yahoo.com", "outlook.com")
    BANK_WORDS = ("bank", "account", "card", "kyc")

    def __init__(self, text, sender="Unknown"):
        self.raw_text = text
        self.sender = sender.strip() or "Unknown"
        self.urls = self.extract_urls()
        self.clean_text = self.preprocess()

    def extract_urls(self):
        found = self.URL_PATTERN.findall(self.raw_text)
        return [u.rstrip(".,;:)!?") for u in found]

    def preprocess(self):
        text = self.raw_text.lower()
        text = re.sub(r"[^a-z0-9\s@./:\-]", " ", text)
        return " ".join(text.split())

    def mentions_bank(self):
        return any(w in self.clean_text for w in self.BANK_WORDS)
```

```
    def get_type(self):
        return "Generic"

    def get_extra_score(self):
        return 0, []


class SMSMessage(Message):
    def get_type(self):
        return "SMS"

    def get_extra_score(self):
        digits = re.sub(r"\D", "", self.sender)
        if len(digits) >= 10 and self.mentions_bank():
            return 2, ["Bank-related SMS sent from an ordinary mobile number (+2)"]
        return 0, []


class EmailMessage(Message):
    def get_type(self):
        return "E-mail"

    def get_extra_score(self):
        domain = self.sender.split("@")[-1].lower()
        if domain in self.FREE_DOMAINS and self.mentions_bank():
            return 2, ["Bank-related e-mail sent from a free e-mail service (+2)"]
        return 0, []
```

17.2 URL Analyzer

```
class URLAnalyzer:
    SHORTENERS = ["bit.ly", "tinyurl.com", "goo.gl", "t.co", "is.gd",
                  "cutt.ly", "rb.gy"]
    BRANDS = {"sbi": "sbi.co.in", "hdfc": "hdfcbank.com",
              "icici": "icicibank.com", "paytm": "paytm.com",
              "amazon": "amazon.in", "flipkart": "flipkart.com",
              "paypal": "paypal.com"}
    BAD_TLDS = (".xyz", ".top", ".tk", ".ml", ".click", ".work", ".buzz")
    IP_PATTERN = re.compile(r"^\d{1,3}(\.\d{1,3}){3}$")

    def analyze(self, url):
        score, reasons = 0, []
        full = url if "://" in url else "http://" + url
        host = (urlparse(full).hostname or "").lower()

        if self.IP_PATTERN.match(host):
            score += 4
            reasons.append(f"{url}: IP address used instead of a domain name (+4)")
        if host in self.SHORTENERS:
            score += 3
            reasons.append(f"{url}: shortened link hides the real site (+3)")
        if "@" in url:
            score += 3
            reasons.append(f"{url}: contains the '@' symbol (+3)")
        if not url.lower().startswith("https://"):
            score += 2
            reasons.append(f"{url}: does not use HTTPS (+2)")
        if host.endswith(self.BAD_TLDS):
            score += 2
            reasons.append(f"{url}: suspicious domain extension (+2)")
        for brand, official in self.BRANDS.items():
            genuine = host == official or host.endswith("." + official)
            if brand in host and not genuine:
                score += 4
                reasons.append(f"{url}: imitates the brand '{brand}' (+4)")
        if host.count("-") >= 3:
            score += 1
            reasons.append(f"{url}: too many hyphens in the domain (+1)")
        return score, reasons
```

17.3 Detector, Result and Log Manager

```
class ScanResult:
    ADVICE = {
        "SAFE": "No major warning signs, but stay careful with unknown links.",
        "SUSPICIOUS": "Do not click links or share details. Verify with the "
                      "organisation using its official app or number.",
        "PHISHING": "Do NOT click any link or share OTP/PIN/passwords. "
```

```
                "Delete the message and report it (cybercrime.gov.in / 1930).",
    }

    def __init__(self, message, score, label, reasons):
        self.msg_type = message.get_type()
        self.sender = message.sender
        self.urls = message.urls
        self.score = score
        self.label = label
        self.reasons = reasons
        self.time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    def display(self):
        print("\n" + "=" * 50)
        print(f"Type : {self.msg_type} Sender: {self.sender}")
        print(f"Links: {', '.join(self.urls) if self.urls else 'None'}")
        print(f"Score: {self.score}")
        print(f"Result : {self.label}")
        print("Reasons:")
        for r in self.reasons:
            print(" -", r)
        print("Advice :", self.ADVICE[self.label])
        print("=" * 50)

    def to_log_line(self):
        return (f"{self.time} | {self.msg_type} | {self.sender} | "
                f"score={self.score} | {self.label} | links={len(self.urls)}\n")


class PhishingDetector:
    KEYWORDS = {"urgent": 2, "immediately": 2, "verify": 2, "refund": 2,
                "click here": 2, "limited time": 2, "otp": 3, "kyc": 3,
                "blocked": 3, "suspended": 3, "prize": 3, "lottery": 3,
                "winner": 3, "password": 3, "account": 1}
    OTP_REQUEST = re.compile(r"(share|send|enter|provide|tell).{0,20}"
                             r"(otp|pin|cvv|password)")
    SAFE_LIMIT = 4
    PHISHING_LIMIT = 10

    def __init__(self):
        self.url_analyzer = URLAnalyzer()

    def classify(self, score):
        if score < self.SAFE_LIMIT:
            return "SAFE"
        elif score < self.PHISHING_LIMIT:
            return "SUSPICIOUS"
        return "PHISHING"

    def scan(self, message):
        score, reasons = 0, []
        text = message.clean_text

        for word, weight in self.KEYWORDS.items():
            if word in text:
                score += weight
                reasons.append(f"Suspicious keyword '{word}' (+{weight})")

        negated = "do not share" in text or "never share" in text
        if self.OTP_REQUEST.search(text) and not negated:
            score += 3
            reasons.append("Asks you to share OTP/PIN/CVV/password (+3)")

        for url in message.urls:
            pts, why = self.url_analyzer.analyze(url)
            score += pts
            reasons.extend(why)

        pts, why = message.get_extra_score() # polymorphic call
        score += pts
        reasons.extend(why)

        if not reasons:
            reasons.append("No phishing indicators found")
        return ScanResult(message, score, self.classify(score), reasons)


class LogManager:
```

```
    def __init__(self, filename="scan_history.txt"):
        self.filename = filename

    def save(self, result):
        try:
            with open(self.filename, "a", encoding="utf-8") as f:
                f.write(result.to_log_line())
        except OSError as e:
            print("Could not save the result:", e)

    def read_history(self):
        try:
            with open(self.filename, "r", encoding="utf-8") as f:
                data = f.read()
            return data if data.strip() else "History is empty."
        except FileNotFoundError:
            return "No history found yet. Scan a message first."
```

17.4 Validation and Main Menu

```
def validate_input(msg_type, text):
    if msg_type not in ("SMS", "EMAIL"):
        raise InvalidMessageError("Message type must be SMS or EMAIL.")
    if not text.strip():
        raise EmptyMessageError("Message cannot be empty.")
    if len(text.strip()) < 10:
        raise InvalidMessageError("Message is too short to analyse.")


def main():
    detector, log = PhishingDetector(), LogManager()
    while True:
        print("\n--- Phishing Message Detection Tool ---")
        print("1. Check Message\n2. View History\n3. Exit")
        choice = input("Enter choice: ").strip()

        if choice == "1":
            try:
                msg_type = input("Type (SMS / EMAIL): ").strip().upper()
                sender = input("Sender (optional): ")
                text = input("Paste the message: ")
                validate_input(msg_type, text)
                cls = SMSMessage if msg_type == "SMS" else EmailMessage
                result = detector.scan(cls(text, sender))
                result.display()
                log.save(result)
            except (EmptyMessageError, InvalidMessageError) as e:
                print("Input error:", e)
            finally:
                print("(Scan finished)")
        elif choice == "2":
            print(log.read_history())
        elif choice == "3":
            print("Stay safe. Goodbye!")
            break
        else:
            print("Please enter 1, 2 or 3.")


if __name__ == "__main__":
    main()
```

17.5 Explanation of the Code

- **Message.__init__**extracts links from the *original* text first, then builds the cleaned text, because cleaning would alter the links.
- **Polymorphism:** the detector calls `message.get_extra_score()` without knowing whether the object is an SMSMessage or an EmailMessage. Python runs the version defined in the proper subclass.
- **URLAnalyzer.analyze** adds a reason string for every point added, so the report is fully explainable.
- **The negation check** ("do not share", "never share") prevents genuine bank warnings such as "never share your OTP" from being penalised for the request pattern.

- **LogManager** uses the `with` statement so the file is always closed, and `FileNotFoundError` is handled so that viewing history before the first scan does not crash the program.
- **try / except / finally** in `main()` catches the custom exceptions and always prints a closing line after a scan attempt.

18. Testing and Test Cases

18.1 Testing Strategy

- **Unit testing:** each class (URLAnalyzer, PhishingDetector, validator) is tested with small inputs.
- **Integration testing:** a full scan from menu input to log file is checked.
- **Boundary testing:** scores just below and at the thresholds (3, 4, 9, 10) are checked.
- **Negative testing:** empty, short and wrongly typed input and a missing log file are tested.

18.2 Test Cases

The expected scores below follow directly from the weights in section 14.5. Run each case on your machine and record the actual output before submission.

TC	Input (type, sender, text)	Expected score	Expected result
1	SMS, no sender, "Hi, the lab meeting is at 5 pm tomorrow."	0	SAFE
2	SMS, no sender, "Your OTP is 482913. Do not share it with anyone."	3 (otp)	SAFE
3	EMAIL, no sender, "Dear customer, verify your account details urgently."	5 (verify 2, urgent 2, account 1)	SUSPICIOUS
4	SMS, no sender, "URGENT! Your SBI account is blocked. Verify KYC immediately at http://sbi-kyc-update.xyz/login"	21 (keywords 13 + no HTTPS 2 + extension 2 + brand 4)	PHISHING
5	SMS, no sender, "You are a lottery winner! Claim prize at http://192.168.10.5/claim"	15 (lottery 3, winner 3, prize 3, IP 4, no HTTPS 2)	PHISHING
6	SMS, no sender, "Click here to track parcel: https://bit.ly/3xYz"	5 (click here 2, shortener 3)	SUSPICIOUS
7	EMAIL, no sender, "Refund ready, open http://amazon.in@fake-site.top"	9 (refund 2, '@' 3, no HTTPS 2, extension 2)	SUSPICIOUS
8	Type = "FAX"	–	InvalidMessageError
9	Empty message	–	EmptyMessageError
10	Message "Hi"	–	InvalidMessageError (too short)
11	View History before any scan	–	"No history found yet" message
12	View History after scans	–	Lines from scan_history.txt

Note on test case 7: the host of that link is *fake-site.top*, so the brand rule does not fire; the '@' rule and the extension rule do.

18.3 Observations from Testing

- Messages with only one or two weak words stay SAFE or SUSPICIOUS, which limits false alarms.
- A message that combines pressure words with a fake-brand link crosses the PHISHING threshold quickly, because the link and text points add up.

- Test case 2 shows the trade-off of a keyword approach: a real OTP alert receives 3 points, and the thresholds are set so that it stays SAFE.

19. Sample Output

Phishing message

```
--- Phishing Message Detection Tool ---
1. Check Message
2. View History
3. Exit
Enter choice: 1
Type (SMS / EMAIL): sms
Sender (optional): 9876543210
Paste the message: URGENT! Your SBI account is blocked. Verify KYC immediately at http://sbi-kyc-
    update.xyz/login

================================================
Type   : SMS   Sender: 9876543210
Links  : http://sbi-kyc-update.xyz/login
Score  : 23
Result : PHISHING
Reasons:
  - Suspicious keyword 'urgent' (+2)
  - Suspicious keyword 'immediately' (+2)
  - Suspicious keyword 'verify' (+2)
  - Suspicious keyword 'kyc' (+3)
  - Suspicious keyword 'blocked' (+3)
  - Suspicious keyword 'account' (+1)
  - http://sbi-kyc-update.xyz/login: does not use HTTPS (+2)
  - http://sbi-kyc-update.xyz/login: suspicious domain extension (+2)
  - http://sbi-kyc-update.xyz/login: imitates the brand 'sbi' (+4)
  - Bank-related SMS sent from an ordinary mobile number (+2)
Advice : Do NOT click any link or share OTP/PIN/passwords. Delete the message and report it
    (cybercrime.gov.in / 1930).
================================================
```

The sender rule (+2) applies here because a mobile number was entered, so the total is 23 instead of the 21 expected for test case 4 without a sender. The order of the keyword lines follows the order of the KEYWORDS dictionary and may differ slightly on your screen.

Safe message

```
Type   : SMS   Sender: Unknown
Links  : None
Score  :  0
Result : SAFE
Reasons:
  - No phishing indicators found
Advice : No major warning signs, but stay careful with unknown links.
```

History view

```
2026-10-23 11:02:14 | SMS | 9876543210 | score=23 | PHISHING | links=1
2026-10-23 11:03:40 | SMS | Unknown | score=0 | SAFE | links=0
```

Error handling

```
Type (SMS / EMAIL): fax
Sender (optional):
Paste the message: Hello there friend
Input error: Message type must be SMS or EMAIL.
(Scan finished)
```

20. Results and Discussion

The design was checked against the test cases in section 18. The expected labels follow from the weighted rules: neutral messages score 0, mild warnings land in the SUSPICIOUS band and messages that combine pressure language with unsafe links reach the PHISHING band. Every verdict is accompanied by the list of rules that caused it, which was the main aim of the project.

The following points were observed during design and testing:

1. **Explainability** – showing the points for each reason makes the verdict easy to trust and to learn from.
2. **Combination effect** – the tool becomes strong when text and link indicators are combined, because phishing messages usually have both.
3. **Tunability** – changing a weight or a threshold in a dictionary or class constant immediately changes the behaviour, without touching the logic.
4. **Robustness** – invalid input and a missing history file are handled without crashing.
5. **Weak point** – keyword matching uses substrings, so a word such as "urgent" also matches "urgently", which is useful, but unrelated words that contain a keyword could add points. Word-boundary matching can be added to improve this.

21. Advantages, Limitations and Future Scope

21.1 Advantages

- Works completely offline and keeps messages private.
- Gives reasons, not just a label, which helps users learn.
- Simple to run: needs only Python and no extra packages.
- Modular, object-oriented and easy to extend.
- Stores scan history for later reference.

21.2 Limitations

- Rule-based; cannot detect completely new attack styles.
- No live blacklist or domain-age check because it is offline.
- English keywords only; no image, attachment or QR-code analysis.
- Genuine messages that use sensitive words may get a few points.
- Console interface only.

21.3 Future Scope

- Add a graphical interface (Tkinter) or a web interface (Flask).
- Add Hindi and Marathi keywords for regional-language phishing.
- Use word-boundary matching and a larger, regularly updated keyword and brand list loaded from a file.
- Add an optional online mode to check links against public blacklists and to check domain age.
- Train a machine-learning classifier (for example Naive Bayes) on a labelled dataset and combine it with the rule score.
- Read messages directly from a mobile notification or e-mail client.
- Export reports as PDF or CSV and show statistics of scans in charts.
- Add a new message class, such as WhatsAppMessage, using the existing inheritance structure.

22. Expected Outcome

- A working Python tool that accepts an SMS or e-mail message and correctly identifies it as Safe, Suspicious or Phishing.
- Clear explanation of the reasons (suspicious words and unsafe links) behind every decision, which helps users learn to spot phishing.
- A saved scan history in a file for later reference.
- A well-structured, reusable and extendable project that demonstrates the main OOP concepts of the course.

23. Tentative Project Plan

Phase / Activity	Tentative Date	Status
Topic Finalization	Week 1	Planned
Problem Definition	Week 2	Planned
Requirement Analysis	Week 3	Planned
System Design	Week 4	Planned
Implementation	Weeks 5–7	Planned
Testing	Week 8	Planned
Final Demonstration	Week 9	Planned

Replace the week numbers with the exact dates given by your faculty coordinator and update the status column as work progresses.

24. Conclusion

The Phishing Message Detection Tool shows how a real-life security problem can be solved with the basic ideas of Python and object-oriented programming. Messages are validated, cleaned and analysed; links are broken into parts and checked; and a weighted risk score gives a clear verdict of SAFE, SUSPICIOUS or PHISHING with reasons that a common user can understand.

The project applies classes, inheritance, polymorphism, composition, regular expressions, string processing, dictionaries and lists, exception handling and file handling in one working application. Because rules are stored as data, the tool is easy to improve. Although a rule-based tool cannot replace full security software, it gives users an instant, private and educational warning before they click a link or share sensitive information, and it can be extended with a graphical interface, regional languages and machine learning in the future.

25. References

1. Python Software Foundation, "Python 3 Documentation", https://docs.python.org/3/
2. Python Software Foundation, "re – Regular expression operations" and "urllib.parse – Parse URLs into components", https://docs.python.org/3/library/
3. Anti-Phishing Working Group (APWG), "Phishing Activity Trends Reports", https://apwg.org/trendsreports/
4. Mark Lutz, "Learning Python", O'Reilly Media.
5. Indian Cyber Crime Coordination Centre, National Cyber Crime Reporting Portal, https://cybercrime.gov.in/
6. Vedashree,"Phishing Message Detection Project Repository", https://vedashree586.github.io/Phishing_message_detection_project/

26. Declaration

We hereby declare that the proposed mini project is our original work and will be developed as part of the Object-Oriented Programming course.

Student Signatures:

1.

2.

Prof. Kavyashree H.N
Faculty Coordinator